

Building a Neural Network from Scratch in NumPy

A staged guide — from one neuron to a full training loop

Chapter 0 — Why This Guide Exists

You already know how to call `sklearn.linear_model.LogisticRegression()`, `torch.nn.Linear(...)`, or `tf.keras.layers.Dense(...)`. This assignment isn't about learning to *use* a neural network — it's about understanding exactly what those libraries are doing underneath, well enough that you could rebuild them yourself.

Each chapter introduces one idea, works through the underlying mathematics explicitly (no hand-waving over what a matrix multiplication actually does), tells you exactly which file to create and what class/method signatures to fill in, and ends with a specific test to run before moving on. **Do not skip ahead if a test is failing.** A bug in Chapter 1 will not announce itself clearly in Chapter 4 — it will look like an unrelated, confusing failure several chapters later. Catch it now, while it is small and isolated.

No solution code appears anywhere in this guide — only method signatures and structure. The mathematics and the implementation are yours to work out.

Documentation Requirements

Every Python file you write is graded on documentation as well as correctness, checked by linting. Specifically:

- Every file starts with a **module-level docstring** — a one- or two-sentence description of what the file contains, as the very first statement in the file, before any imports.
- Every class has a **class docstring** describing what it represents.
- Every method has a **docstring** describing what it does, its parameters, and what it returns — including `__init__`.

The linter is **ruff**, already listed in `environment.yml` (Chapter 0.4). Run it from the submission root:

A terminal window with a dark background. The title bar shows three colored dots (red, yellow, green) on the left, the word "terminal" in the center, and "[copy]" on the right. The command "ruff check nn/" is entered in green text on a black background.

```
terminal [copy]
ruff check nn/
```

A **ruff** configuration file (`ruff.toml` or `pyproject.toml`, depending on how it is provided) is included in the assignment repository and pins the exact rule set — the **pydocstyle** D rules under the "google" convention, which is the same **Args:/Returns:** style every stub in this guide already

uses. **You do not configure the linter yourself** — do not add your own `ruff` config, and do not change the provided one.

The code stub shown for `nn/module.py` below already follows this format — match it in every file you create from here on, including the ones with no example shown. A method that only contains `pass` or `raise NotImplementedError` still needs its docstring written *before* you implement the body — writing the docstring first is also a reliable way to clarify what a method is actually supposed to do before you start coding it.

This is a standing rule for the rest of the guide, not just this chapter: every “Validate before moving on” section from here on requires `ruff check nn/` to exit 0, in addition to that chapter’s `pytest` command — both are checked, not just the tests.

Naming Conventions

Use these exact names everywhere — in your code, and in how you think about the math. Every chapter, every stub, and every worked example in this guide follows them without exception:

- W — weight matrix. b — bias vector.
- dW , db — their gradients, stored as `self.dW`, `self.db`.
- x — a layer’s input. z — its pre-activation output (before the activation function). a — its post-activation output (after the activation function).
- `parameters()` returns `[(self.W, self.dW), (self.b, self.db)]`.

If you see a different letter used informally somewhere (a lowercase w , for instance), it refers to one specific piece of W — never a different object with its own meaning.

What’s Provided vs. What You Build

The assignment repository you were given already contains four things: `environment.yml` (the exact, fixed set of dependencies — Chapter 0.4), the `tests/` directory (the specification your code must satisfy, one test file per chapter below), the dataset, and this guide. You do not create, modify, or move any of those. **What you build is everything under `nn/`, plus `main.py`** — nothing else in the repository is yours to write.

Chapter 0.4 — Environment Setup

One environment, already specified

This project uses exactly one environment, managed by `conda`, defined entirely by the `environment.yml` provided in the assignment repository. You do not hand-install packages one at a time, and you do not write your own dependency file — `environment.yml` is provided and already correct.

Prerequisite: `conda` (Miniconda or Anaconda). If running `conda` at a terminal gives “command not found,” install Miniconda first: <https://www.anaconda.com/docs/getting-started/installation>.

Setup — run once, from the repository root

```
conda env create -f environment.yml
conda activate iml-formative1
```

This environment provides Python 3.11 with NumPy (2.x) and `pytest` — and nothing else. No deep learning framework (PyTorch, TensorFlow, JAX, ...) is installed, and using one is not permitted: not relying on one is the entire point of this assignment. If an import works in some other environment on your machine but not here, that is this environment doing its job correctly, not a bug to work around.

Every new terminal starts fresh. Run `conda activate iml-formative1` again in any new terminal window or session before running any code or test in this project — the environment does not stay active automatically between sessions.

Validate before moving on

Run:

```
python -c "import numpy, pytest; print('numpy', numpy.__version__)"
pytest --version
```

Both commands must succeed without error. Expect a NumPy version starting with 2. and a pytest version starting with 8. — if you see a different major version for either, you are very likely not in the `iml-formative1` environment; re-run `conda activate iml-formative1` and try again.

Do not proceed to Chapter 0.5 until both commands succeed.

If the provided dependency list ever changes over the course of the assignment, re-sync your environment rather than installing the difference by hand:

```
conda env update -f environment.yml --prune
```

Chapter 0.5 — Project Setup

Before Chapter 1, set up your **submission root** — the single directory containing `nn/`, `tests/`, and `main.py`. Every `pytest ...` and `python main.py` command anywhere in this guide is run *from that directory*, with the `iml-formative1` environment active (Chapter 0.4). The tree below shows it under the example name `your_submission/` — create it now:

```
terminal [copy]

your_submission/
  nn/
    __init__.py
    module.py
    layers/
      __init__.py
      linear.py
    activations/
      __init__.py
      relu.py
      sigmoid.py
      softmax.py
    losses/
      __init__.py
      cross_entropy_loss.py
      categorical_cross_entropy_loss.py
    optim/
      __init__.py
      sgd.py
  tests/                                (provided -- do not edit)
  data/                                (provided -- do not edit)
  main.py
  environment.yml                      (provided -- do not edit)
  README.md
```

`environment.yml` (Chapter 0.4) and the dataset are also provided in the repository, alongside `tests/` — you do not create, download, or place either yourself.

Always run `pytest` (and `python main.py`) from the submission root, with the environment active, so that `import nn...` resolves correctly. If you see `ModuleNotFoundError: No module named 'nn'`, you are either in the wrong directory or the wrong environment — `cd` back to the submission root and confirm `conda activate iml-formative1` succeeded first.

This is deliberately small: one linear layer, three activations (ReLU, sigmoid, softmax), two losses (binary and categorical cross-entropy), one optimizer (SGD), and `main.py` to wire a chosen combination together and train on real data. The network itself stays a single layer throughout — no container class chaining multiple layers. With only one layer, you call it and its activation directly, one after another, in `main.py` itself. That is simpler to write and, more importantly, simpler to debug: every value that flows through your network passes through code you can see in one file.

Every `__init__.py` file starts with **just a one-line docstring** describing the subpackage, and nothing else — `ruff`'s documentation check applies to packages too, so a truly empty file will fail it. For example, `nn/activations/__init__.py` starts as:

```
Python

"""Activation function implementations."""
```

You will add one import line below that docstring as you complete each corresponding class, so that `from nn.activations import ReLU` works instead of the longer `from nn.activations.relu`

import ReLU. You will be told exactly which line to add at the point you need it.

Start with `nn/module.py`. This defines the shared contract every layer, activation, and loss follows.

Step 1. Create the file `nn/module.py`.

Step 2. Inside it, define the base class exactly as shown below — method signatures and structure only, nothing to implement yet:

Python

```
"""Base class defining the shared interface for every layer, activation, and
loss."""
```

```
class Module:
```

```
    """Base class every layer and activation subclasses.
```

```
    Defines the forward/backward contract, plus default
    (empty) parameter and gradient-reset behavior for
    modules that have no learnable weights.
    """
```

```
    def forward(self, x):
```

```
        """Compute this module's output given input x.
```

```
        Args:
```

```
            x: input array.
```

```
        Returns:
```

```
            The module's output for this input.
```

```
        """
```

```
        raise NotImplementedError
```

```
    def backward(self, grad_output):
```

```
        """Compute gradients given the upstream gradient.
```

```
        Args:
```

```
            grad_output: gradient of the loss with respect
                        to this module's output.
```

```
        Returns:
```

```
            Gradient of the loss with respect to this
            module's input.
```

```
        """
```

```
        raise NotImplementedError
```

```
    def parameters(self):
```

```
        """Return this module's learnable parameters.
```

```
        Returns:
```

```
            A list of (param, grad) pairs. Empty for
            modules with no learnable weights.
```

```

    """
    return []

def zero_grad(self):
    """Reset any stored gradients to zero.

    No-op by default. In this project, resetting stored
    gradients is handled by the optimizer (Chapter 9)
    directly, not by individual modules -- you will not
    need to override this method anywhere, including in
    Linear.
    """
    pass

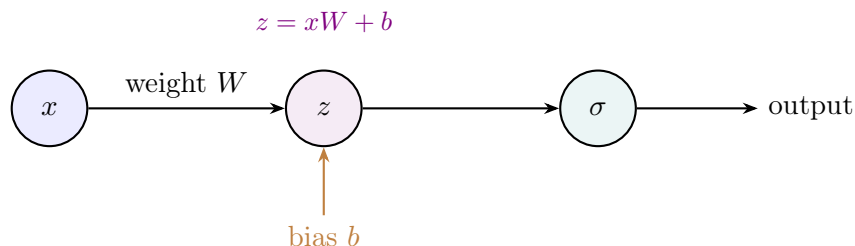
```

Step 3. Save the file. You will not edit `module.py` again — every class you write from here on subclasses `Module` and overrides `forward/backward` with real logic. Some (like activations) will not need to override `parameters()` or `zero_grad()` at all, because they have no learnable weights — the base class’s defaults already handle that correctly.

Chapter 1 — One Neuron, Many Inputs

The mental model

Before any matrices: a neuron does three things to its input — **scales** it by a learned weight, **shifts** it by a learned bias, then **squashes** the result through a nonlinear activation function σ .



With a single input, that is $z = xW + b$, then $\text{output} = \sigma(z)$ — nothing more. If you have used `sklearn.linear_model.LogisticRegression` with one feature, you have already used exactly this. Hold that picture in your head; you are about to generalize it.

The idea: from one number to a vector of inputs

Real inputs are rarely a single number — a neuron usually receives several input features at once. Say you have n input features. You now need **one weight per input feature**, not just one weight total.

Your input is a vector, and your weights form a matrix W with a single column — one weight per feature, shape $(n, 1)$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad \text{and} \quad W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \quad \text{shape } (n, 1).$$

Here $w_1, \dots, w_n$ denote the individual entries of W 's one column — not a separate object from W , just how you refer to one piece of it.

There is a specific operation that takes an input vector and a weight matrix and combines them into a *single number*. Written out in full, for $n = 3$, that operation is:

$$z = xW + b = w_1x_1 + w_2x_2 + w_3x_3 + b.$$

This is the calculation to work through by hand before writing any code. Notice what happened to the dimensions: a vector and a matrix went in, and *one scalar* came out. That collapsing to a single number is the entire content of this operation — if your code's output is still a vector after this step, you have not done this multiplication correctly, you have done something else (elementwise multiplication is the usual culprit — it keeps the vector shape instead of collapsing it).

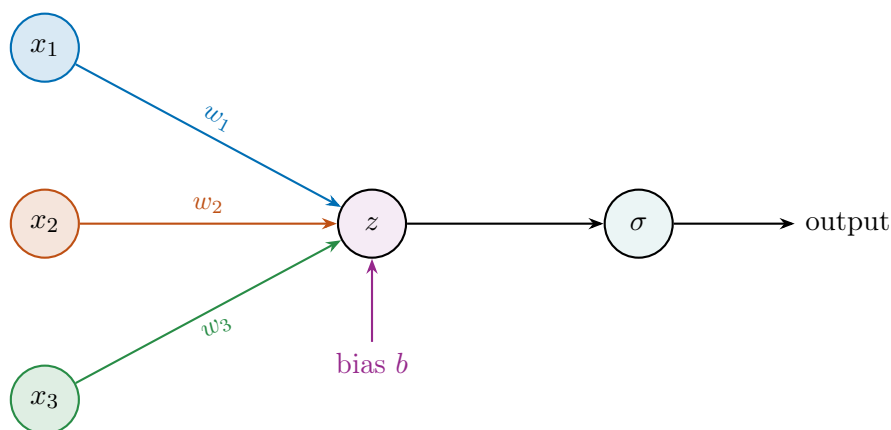
The same operation, written as a matrix multiplication

This *is already* a matrix multiplication — no separate “vector version” to translate from. Lay the input out as a 1×3 *row* (x), and W is already a 3×1 *column* by construction. Multiply them the

same way you would multiply any matrices — row against column, multiply matching positions, sum the results:

$$xW = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} = w_1x_1 + w_2x_2 + w_3x_3.$$

The shapes tell you the result before you compute a single value: (1×3) multiplied by (3×1) leaves the two outer dimensions, 1×1 — a single number. This shape bookkeeping is the habit that will save you the most debugging time later: before running any matrix multiplication, write down the two shapes involved and cancel the matching inner dimension to predict the output shape, the same way you just did here. Notice also that this is exactly `x @ self.W` — `x` on the left, `W` on the right, no transpose anywhere — which is why the code stub you will write later has that exact order.



Worked numeric example — do this one by hand first

Let

$$\mathbf{x} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}, \quad W = \begin{bmatrix} 0.5 \\ 1.0 \\ -0.5 \end{bmatrix}, \quad \text{shape } (3, 1), \quad b = 0.2.$$

Multiply each matching pair first, keeping the colors so you can trace which input paired with which weight, then sum everything at the end:

$$w_1x_1 = 0.5 \cdot 2 = 1.0$$

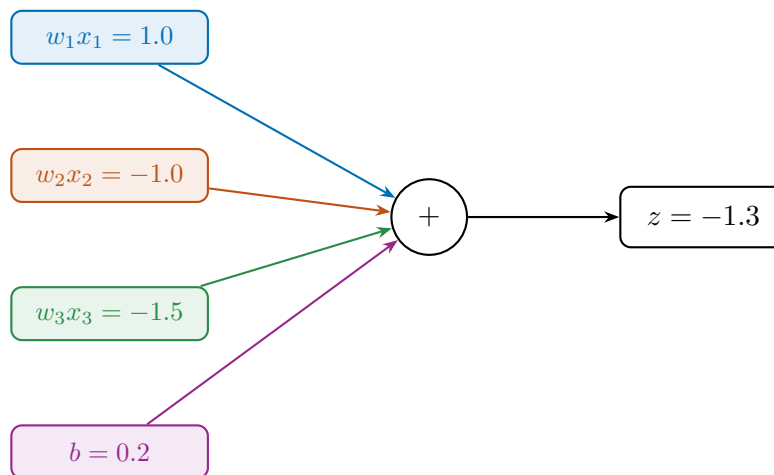
$$w_2x_2 = 1.0 \cdot -1 = -1.0$$

$$w_3x_3 = -0.5 \cdot 3 = -1.5$$

$$z = 1.0 + (-1.0) + (-1.5) + 0.2 = \boxed{-1.3}.$$

Here is the same computation as a diagram, so you can see the three colored terms physically

flow into the sum and come out as one number:



Compute this yourself, by hand, before you touch NumPy. Once your code agrees with -1.3 on this exact input, you know your **forward** is doing the right operation — not just producing *a* number, but producing *this specific* number.

A quick note before you build: if your input has n features, you need exactly n weights — one per feature, since each w_i only ever multiplies its matching x_i . And the bias stays a single number, not a vector: look at the worked example above — b appears once in the sum, added after the multiplication collapses everything else to a scalar, not once per feature.

The multiplication and the bias are two separate steps — don't merge them in your head. NumPy gives you more than one way to compute xW itself (before b is added at all). The code you write later is `x @ self.W` specifically — x first, then W , in that order, since matrix multiplication is not commutative and this is the order that produces the right output shape:

- `np.dot(x, W)` — computes it directly, in one call.
- `x @ W` — the same result, using Python's matrix-multiplication operator. This is the form you will actually use in `Linear.forward`.
- `np.sum(x * W.T)`, if you insist on the elementwise-then-sum route — but note this needs `W.T` to line up shapes for elementwise multiplication in the first place, which is exactly the extra fuss `@` avoids. Prefer `@`; this option is here mainly so you recognize what it looks like if you see it in someone else's code.

All of these give the identical scalar (or, for a batch, the identical column of scalars). Only *after* you have that result do you add b — a plain, ordinary addition, not part of the multiplication itself. Confirm your result agrees with -1.3 above once b is added.

If you want to see exactly what each function does beyond this guide's description, the official documentation is thorough and has runnable examples: `np.dot` (<https://numpy.org/doc/stable/reference/generated/numpy.dot.html>), `@` / `np.matmul` (<https://numpy.org/doc/stable/reference/generated/numpy.matmul.html>), and `np.sum` (<https://numpy.org/doc/stable/reference/generated/numpy.sum.html>).

Where this is headed: from one example to a batch

Everything above processes **one** example at a time — one $\mathbf{x}$ in, one z out. In practice, you almost never do this: training and prediction both run on many examples at once, stacked together into a *batch*. This is the form you will actually implement and use, so it is worth seeing now, even though the file you build in this chapter still handles one example at a time.

Instead of a single input vector $\mathbf{x}$, a batch of m examples is a *matrix* X with one row per example:

$$X = \begin{bmatrix} \text{---} & \mathbf{x}^{(1)} & \text{---} \\ \text{---} & \mathbf{x}^{(2)} & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}^{(m)} & \text{---} \end{bmatrix}, \quad X \text{ has shape } (m, n) \text{ for } n \text{ features.}$$

The single multiplication xW becomes XW : multiply the $(m \times n)$ matrix by W 's $(n, 1)$ shape, and you get one z value *per row* back — an $(m, 1)$ matrix, all m examples processed in a single operation instead of a loop:

$$XW + b = \begin{bmatrix} z^{(1)} \\ z^{(2)} \\ \vdots \\ z^{(m)} \end{bmatrix}, \quad \underbrace{(m \times n)}_X \cdot \underbrace{(n, 1)}_W \longrightarrow \underbrace{(m, 1)}_{\text{one } z \text{ per row}}.$$

Small worked example with $m = 2$ examples, $n = 3$ features each, reusing the same W and b from above:

$$X = \begin{bmatrix} 2 & -1 & 3 \\ 1 & 0 & -2 \end{bmatrix}, \quad W = \begin{bmatrix} 0.5 \\ 1.0 \\ -0.5 \end{bmatrix} \Rightarrow XW + b = \begin{bmatrix} -1.3 \\ 1.7 \end{bmatrix}, \quad \text{shape } (2, 1).$$

Row 1 is exactly the -1.3 you already computed by hand. Row 2 is the same operation applied to a second example, computed at the same time — nothing new mathematically, just the same multiplication repeated once per row. A later chapter has you extend `Linear.forward` to accept X instead of a single $\mathbf{x}$; when you get there, this is the shape transformation you are implementing.

What to build

This is your first permanent file.

Step 1. Create the file `nn/layers/linear.py`.

Step 2. Inside it, import the base class and define `Linear` as a subclass, with these four methods stubbed out exactly as shown — one blank line between each method, matching standard Python style:

```
Python

"""Linear (fully connected) layer: z = xW + b."""

import numpy as np

from nn.module import Module
```

```

class Linear(Module):
    """A fully connected layer computing  $z = xW + b$ .

    Attributes:
        W (np.ndarray): weight matrix, shape
            (in_features, out_features).
        b (np.ndarray): bias vector, shape (out_features,).
    """

    def __init__(self, in_features: int, out_features: int) -> None:
        """Initialize the layer's weights and bias.

        Args:
            in_features (int): number of input features.
            out_features (int): number of output neurons.

        Sets:
            self.W (np.ndarray): weight matrix, shape
                (in_features, out_features). Xavier-initialized,
                not zeros (see "Weight initialization" below).
            self.b (np.ndarray): bias vector, shape
                (out_features,). Initialized to zero.
        """
        # TODO: create self.W and self.b as described above
        pass

    def forward(self, x: np.ndarray) -> np.ndarray:
        """Compute this layer's output for a batch of inputs.

        Args:
            x (np.ndarray): input, shape (batch_size, in_features).

        Returns:
            np.ndarray: output, shape (batch_size, out_features).
        """
        # TODO: implement the forward pass, using self.W
        # and self.b
        pass

    def backward(self, grad_output: np.ndarray) -> np.ndarray:
        """Compute gradients given the upstream gradient.

        Args:
            grad_output (np.ndarray): gradient of the loss with
                respect to this layer's output, shape
                (batch_size, out_features).

        Returns:
            np.ndarray: gradient of the loss with respect to
                this layer's input, shape

```

```

        (batch_size, in_features).

    Sets:
        self.dW (np.ndarray): gradient of the loss with
            respect to self.W, same shape as self.W.
        self.db (np.ndarray): gradient of the loss with
            respect to self.b, same shape as self.b.
    """
    # TODO: implement in a later chapter -- leave as-is
    pass

def parameters(self) -> list[tuple[np.ndarray, np.ndarray]]:
    """Return this layer's learnable parameters.

    Returns:
        list[tuple[np.ndarray, np.ndarray]]: pairs of
            (parameter, gradient) --
            [(self.W, self.dW), (self.b, self.db)].
    """
    # TODO: implement in a later chapter -- leave as-is
    pass

```

Step 3. For this chapter, only implement `__init__` and `forward`, and only for the single-example case shown above. Use exactly `self.W` and `self.b` as your attribute names – the test suite and later chapters (backward, the optimizer) look for a layer’s parameters under those exact names. Leave `backward` and `parameters` as `pass` – you will fill those in during a later chapter on backpropagation. You will also come back to extend `__init__`/`forward` themselves in the next two chapters (multiple neurons, then batching) — this stays the same file throughout, built up incrementally, not replaced.

Step 4. Once `forward` works, open `nn/layers/__init__.py` and add this one line:

```

Python

from .linear import Linear

```

This is what makes `from nn.layers import Linear` work elsewhere, instead of the longer `from nn.layers.linear import Linear`.

Weight initialization: what actually goes in `self.W`

The stub above says “create `self.W`” but not what values to put in it. Unlike the forward-pass tests elsewhere in this guide, `test_stage1_vector.py` checks your `__init__` directly, not just your `forward` output — so this is not optional groundwork you can defer, it is graded in this chapter. See “Validate before moving on” below for exactly what is checked.

Never initialize W to all zeros. With more than one neuron (Chapter 2), an all-zero W makes every neuron compute the exact same output from the same input and receive the exact same gradient during backpropagation — they never differentiate from each other, no matter how long you train. This is called the **symmetry problem**. It does not actually bite yet in this chapter,

since a single neuron has no other neuron to be symmetric *with* — but the habit you build now is what Chapter 2 depends on, so start correctly here rather than relying on “it happens to work with one neuron.” (The bias b is the one exception: it is conventionally initialized to zero, in both this chapter and the next — symmetry-breaking is specifically a W problem.)

The fix is small *random* values, not zeros. A shortcut you may have seen elsewhere is multiplying random values by a fixed constant, e.g. `np.random.randn(...) * 0.01`. That specific constant is not wrong for a network this small — it mainly causes problems in *deep* networks (many stacked layers), where a fixed small scale compounds across layers, and this project never has more than one layer. Still, an arbitrary constant does not adapt to a layer’s actual size, and there is a standard, principled alternative worth learning instead of a magic number:

Xavier (Glorot) initialization scales the random range using the layer’s own shape, so output variance stays reasonable regardless of how many inputs or outputs the layer has. For a layer with `in_features` and `out_features`, draw each weight uniformly from

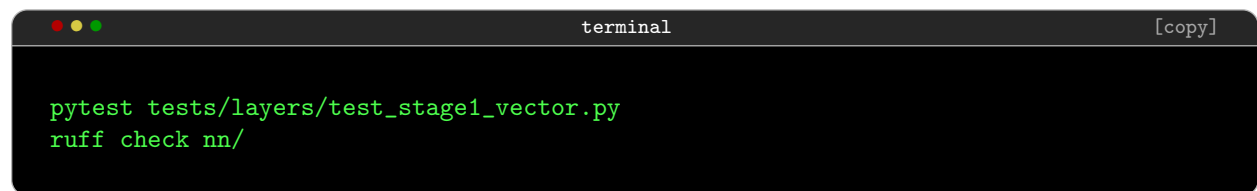
$$W_{ij} \sim \text{Uniform}\left(-\sqrt{\frac{6}{\text{in_features} + \text{out_features}}}, \sqrt{\frac{6}{\text{in_features} + \text{out_features}}}\right).$$

This is the standard formula, and is well-suited to sigmoid/softmax-family activations specifically (what this project uses). `np.random.uniform` takes a low and high bound directly, so this translates fairly directly into code — work out what the bound evaluates to for a given `in_features/out_features` pair by hand, and confirm your implementation matches, the same way you have confirmed every other worked example in this guide.

You do not need a separate method for this — initialize W and b directly inside `__init__`, the same place you already store them. **This chapter’s test checks your actual initialization** (see below); *later* chapters’ tests move on from that and instead set `layer.W/layer.b` directly to known worked-example values before checking `forward/backward` — at that point your random initialization is bypassed for the sake of a reproducible test, but it still has to be correct here first.

Validate before moving on

Run both of these — both must exit 0 before you proceed:

A terminal window with a dark background and light green text. The title bar shows three colored dots (red, yellow, green) on the left and the word "terminal" in the center. On the right side of the title bar is a "[copy]" button. The terminal content shows two lines of code: `pytest tests/layers/test_stage1_vector.py` and `ruff check nn/`.

```
terminal [copy]
pytest tests/layers/test_stage1_vector.py
ruff check nn/
```

`pytest` here checks more than `forward`. `test_stage1_vector.py` also checks your `__init__` directly:

- `layer.W` has shape `(in_features, out_features)` and a float dtype.
- `layer.W` is **not** all zeros.
- `layer.W`’s values are consistent with the Xavier uniform bound for the given `in_features/out_features` — both the bound and the spread of values are checked, within a stated tolerance.
- `layer.b` has shape `(out_features,)` and **is** all zeros.

A zeros initialization, or an `np.random.randn(...)` * 0.01 initialization, will fail this test — not just be non-standard. Get the actual Xavier bound right, not just “some small random numbers.”

`ruff check nn/` checks the documentation requirements from Chapter 0 — module/class/method docstrings, in the Google convention — against every file under `nn/`, not just `linear.py`.

Beyond `__init__`, this same test file also checks the **shape** of your **forward** output and the **value**, against the worked example above and others like it. The expected output shape is $(1, 1)$ — a 1×1 array, **not** a bare Python float. If you call `.item()` on your result, or otherwise index/flatten it down to a plain number before returning, the shape check will fail even though the value is correct.

Do not proceed to Chapter 2 until both commands exit 0.

Chapter 2 — Many Neurons: the Weight Matrix, and Softmax

The idea

Chapter 1 built one neuron: one weight column, one bias, one output z per example — and `Linear`’s constructor already takes `in_features` and `out_features` as separate arguments, so it already supports any number of output neurons. Classification with C classes needs C output neurons, one per class, each scoring the same input independently. Chapter 1’s single neuron was the $C = 1$ case the whole time.

Stack the C weight columns side by side as the **columns** of W :

$$W = \begin{bmatrix} w_1^{(A)} & w_1^{(B)} & w_1^{(C)} \\ w_2^{(A)} & w_2^{(B)} & w_2^{(C)} \\ w_3^{(A)} & w_3^{(B)} & w_3^{(C)} \end{bmatrix}, \quad W \text{ has shape } (n, C), \quad b = [b_A \quad b_B \quad b_C], \text{ shape } (C,).$$

This is exactly Chapter 1’s W , generalized only in its number of columns: Chapter 1 used `out_features=1`, one column; here `out_features=C`, C columns.

The logits are computed exactly the same way as Chapter 1 — no new formula, same $z = xW + b$:

$$\text{one example: } \underbrace{(1 \times n)}_x \cdot \underbrace{(n \times C)}_W \longrightarrow \underbrace{(1 \times C)}_z, \quad \text{a batch: } \underbrace{(m \times n)}_X \cdot \underbrace{(n \times C)}_W \longrightarrow \underbrace{(m \times C)}_Z.$$

Spell out one entry to see what each number actually means:

$$Z_{i,c} = (\text{row } i \text{ of } X) \cdot (\text{column } c \text{ of } W) + b_c$$

— example i scored against class c ’s weight column. This is exactly C independent Chapter-1 dot products, computed together in one matrix multiplication: column c of W is class c ’s own Chapter-1 weight vector, doing Chapter 1’s exact operation against every row of X at once.

These C numbers per example are called **logits** — one score per class, not yet probabilities (they can be negative, and do not sum to 1). Closing that gap is softmax’s job.

Worked example — extending Chapter 1’s numbers

Reuse $\mathbf{x} = [2, -1, 3]$ from Chapter 1. Column A below is exactly Chapter 1’s weight column, now sitting inside a larger W alongside two more classes:

$$W = \begin{bmatrix} 0.5 & -0.3 & 0.1 \\ 1.0 & 0.2 & -0.6 \\ -0.5 & 0.4 & 0.3 \end{bmatrix}, \quad b = [0.2 \quad -0.1 \quad 0.05].$$

$$xW + b = [-1.3 \quad 0.3 \quad 1.75].$$

Column A is still Chapter 1’s $[0.5, 1.0, -0.5]$, so $z_A = -1.3$ again, exactly as before. Verify z_B and z_C by hand the same way you verified z_A in Chapter 1, before moving on.

Softmax: turning logits into probabilities

Logits are not yet a valid probability distribution. Softmax fixes both problems at once — exponentiate every logit (forces every value positive), then divide each by the sum of all of them (forces the total to 1):

$$\text{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}.$$

Applied to $\mathbf{z} = [-1.3, 0.3, 1.75]$:

$$e^{\mathbf{z}} = [0.2725, 1.3499, 5.7546], \quad \sum_k e^{z_k} = 7.3770.$$

$$\text{softmax}(\mathbf{z}) = [0.0369, 0.1830, 0.7801], \quad \text{sum} = 1.0000.$$

Class *C* had the highest logit (1.75) and gets the highest probability (0.78) — softmax preserves the *ordering* of the logits, it just reshapes them into something that sums to 1 and can be interpreted as confidence.

Numerical stability — do not skip this

Exponentiating a large logit overflows: e^{1000} is already too large for a float to represent, and `np.exp` will silently produce `inf`, followed by `nan` once you divide. The fix is to **subtract the maximum logit from every logit before exponentiating**:

$$\text{softmax}(\mathbf{z})_c = \frac{e^{z_c - \max_k z_k}}{\sum_k e^{z_k - \max_k z_k}}.$$

This is not an approximation — it gives *exactly* the same result, because the same constant is subtracted from every term, top and bottom, and cancels out algebraically. Confirm this yourself on the worked example above: subtract $\max(\mathbf{z}) = 1.75$ from every logit first ($\mathbf{z}$ becomes $[-3.05, -1.45, 0]$), then exponentiate and normalize — you should get the identical $[0.0369, 0.1830, 0.7801]$. The only difference is that every exponent is now ≤ 0 , so every $e^{(\cdot)} \leq 1$: no overflow is possible, no matter how large the original logits were.

Computing this for a batch: axis and keepdims

The formula above is written per-example, as if $\mathbf{z}$ were a single row. Your actual input is a batch Z of shape (m, C) — m examples, each with C logits — and `np.max/np.sum` need two extra arguments to do the right thing on a 2D array instead of collapsing it into one number.

axis=1. By default, `np.max(Z)` (no `axis` argument) returns a single scalar — the largest value anywhere in the whole batch. That is not what you want: each example needs *its own* maximum, computed only across *its own* C logits, not against every other example’s logits too. `axis=1` tells NumPy “reduce across the columns, separately for each row” — exactly one max per example. The same reasoning applies to the denominator’s sum: `np.sum(..., axis=1)`, not a single sum over the entire batch.

keepdims=True. `np.max(Z, axis=1)` on its own returns shape $(m,)$ — a flat list of m maxima, one dimension collapsed away. That shape does *not* subtract cleanly from Z ’s shape (m, C) : NumPy’s broadcasting rules align shapes from the *trailing* dimension, so a $(m,)$ array broadcasts against Z ’s last axis (size C), not its first (size m). If $m \neq C$, this raises a broadcasting error you will see immediately. If m happens to *equal* C — entirely possible, e.g. a batch of 3 examples with 3 classes — there is no error at all: NumPy silently broadcasts against the wrong axis, subtracting example 1’s max from class 1’s column instead of from example 1’s row, and you get a number back that looks plausible but is simply wrong. This is the more dangerous failure mode precisely because nothing crashes. `keepdims=True` keeps the reduced axis as size 1 instead of removing it — shape $(m, 1)$ instead of $(m,)$ — which broadcasts correctly against (m, C) no matter what C is, and does so identically for both `np.max` and the denominator’s `np.sum`.

If you want to see this behavior directly rather than take it on faith, the official documentation covers both parameters with runnable examples: `np.max` (<https://numpy.org/doc/stable/>

`reference/generated/numpy.amax.html`) and `np.sum` (`https://numpy.org/doc/stable/reference/generated/numpy.sum.html`) — both list `axis` and `keepdims` as parameters with the same meaning described here.

A shape subtlety: softmax is not fully elementwise

ReLU or sigmoid compute each output using only the matching input — output i depends only on input i . Softmax is different: every output depends on *every* logit in that row, through the shared denominator $\sum_k e^{z_k}$. The *shape* is still preserved (a length- C vector in, a length- C vector out; a batch of shape (m, C) in, (m, C) out) — but do not implement it with the same per-element independence you might assume from other activations. For a batch, the sum in the denominator must be computed **per row**, not over the whole batch at once.

What to build

Your `Linear` class already does this. It takes `in_features` and `out_features` as separate arguments and builds `self.W` with shape `(in_features, out_features)`, so `Linear(n, C)` is already C neurons sharing one input. `forward`'s `x @ self.W + self.b` already returns (m, C) for a batch of m examples — C logits per example. **There is nothing to add to `linear.py` here.** A dedicated test below confirms it directly — see “Validate before moving on.”

Step 1. Create the file `nn/activations/softmax.py`:

```
Python

"""Softmax activation: converts logits into a probability distribution."""

import numpy as np

from nn.module import Module

class Softmax(Module):
    """Softmax activation, applied row-wise to a batch of logits.

    Unlike ReLU or Sigmoid, each output depends on every logit in
    its own row, not just the matching input -- see "A shape
    subtlety" above.
    """

    def forward(self, x: np.ndarray) -> np.ndarray:
        """Compute softmax probabilities for a batch of logits.

        Args:
            x (np.ndarray): logits, shape (batch_size, C).

        Returns:
            np.ndarray: probabilities, shape (batch_size, C).
            Each row sums to 1.
        """
        # TODO: implement, using the max-subtraction
        # trick above for numerical stability
```

```

pass

def backward(self, grad_output: np.ndarray) -> np.ndarray:
    """Compute gradients given the upstream gradient.

    Args:
        grad_output (np.ndarray): gradient of the loss with
            respect to this layer's output, shape
            (batch_size, C).

    Returns:
        np.ndarray: gradient of the loss with respect to
            this layer's input (the logits), shape
            (batch_size, C).
    """
    # TODO: implement in a later chapter -- leave as-is
    pass

```

Note there is no `__init__` to write and no `parameters()` override needed — Softmax has no learnable weights, so the Module base class's default (an empty list) is already correct.

Step 2. Add this line to `nn/activations/__init__.py`:

```

Python

from .softmax import Softmax

```

Validate before moving on

There is nothing new to *add* to `linear.py` — your `Linear` class already supports $C > 1$ unchanged from Chapter 1. Confirm it with the dedicated multi-class check:

```

terminal [copy]

pytest tests/layers/test_stage2_matrix.py

```

This builds `Linear(n, C)` with $C > 1$ and confirms `forward` returns shape (m, C) correctly — it is testing that your existing code generalizes, not asking you to write anything new.

What *is* new this chapter is Softmax. Run:

```

terminal [copy]

pytest tests/activations/test_stage2_softmax.py
ruff check nn/

```

The `pytest` run checks softmax's output always sums to 1, matches the worked example's values, and — critically — does not produce `nan` or `inf` on deliberately large logits. That last check is

where an unstable implementation gets caught; passing the value check on small numbers does not mean your implementation is stable.

Do not proceed until all three pass.

Chapter 3 — Backpropagation Through Linear

The idea

Every module you have built so far only computes **forward**. Training needs the reverse direction too: given the gradient of the loss with respect to this layer's *output* (called **grad_output**, shape matching Z), compute the gradient of the loss with respect to this layer's *parameters* (W , b) and with respect to its *input* (X) — so the layer before this one can continue the chain.

Three gradients, three shapes to get right:

$$\frac{\partial L}{\partial W} \text{ must match } W\text{'s shape } (n, C), \quad \frac{\partial L}{\partial b} \text{ must match } b\text{'s shape } (C,), \quad \frac{\partial L}{\partial X} \text{ must match } X\text{'s shape } (m, n).$$

The three formulas

For $Z = XW + b$ — the same formula from Chapters 1–2, no transpose anywhere in forward — with **grad_output** = $\frac{\partial L}{\partial Z}$ of shape (m, C) :

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Z}, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^m \left(\frac{\partial L}{\partial Z} \right)_{i,:}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^\top.$$

Check the shapes before trusting the formulas: $(n \times m) \cdot (m \times C) \rightarrow (n, C)$ for $\partial L / \partial W$ — matches W . Summing **grad_output** over its rows (the batch axis) gives a single $(C,)$ vector — matches b . And $(m \times C) \cdot (C \times n) \rightarrow (m, n)$ for $\partial L / \partial X$ — matches X . Notice which side gets transposed: X transposes for $\partial L / \partial W$ (so it can multiply against **grad_output** correctly), and W transposes for $\partial L / \partial X$ — the reverse of forward's plain XW , which is exactly what you would expect running the computation backward.

This is the single most common bug in the whole assignment. $\partial L / \partial W$ and $\partial L / \partial b$ are *summed over the batch* — every example contributed to the same shared W and b , so their gradients accumulate. $\partial L / \partial X$ is **not** summed — each example has its own row of X , and needs its own separate gradient row back. If you sum when you should not (or vice versa), your code will often still run without crashing — it will just train wrong, silently. This is exactly what gradient checking (below) is for.

Worked example

Reuse the batch of 2 examples from Chapter 1's preview, with a single output neuron ($W = [0.5, 1.0, -0.5]^\top$, shape $(3, 1)$, $b = 0.2$, so $Z = [-1.3, 1.7]$ for the two rows), and a made-up upstream gradient (as if it came from a loss further downstream):

$$X = \begin{bmatrix} 2 & -1 & 3 \\ 1 & 0 & -2 \end{bmatrix}, \quad \frac{\partial L}{\partial Z} = \begin{bmatrix} 0.4 \\ -0.9 \end{bmatrix} \quad (\text{shape } (2, 1)).$$

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Z} = \begin{bmatrix} 2 & 1 \\ -1 & 0 \\ 3 & -2 \end{bmatrix} \begin{bmatrix} 0.4 \\ -0.9 \end{bmatrix} = \begin{bmatrix} -0.1 \\ -0.4 \\ 3.0 \end{bmatrix}, \quad \frac{\partial L}{\partial b} = [0.4 + (-0.9)] = [-0.5].$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^\top = \begin{bmatrix} 0.4 \\ -0.9 \end{bmatrix} \begin{bmatrix} 0.5 & 1.0 & -0.5 \end{bmatrix} = \begin{bmatrix} 0.2 & 0.4 & -0.2 \\ -0.45 & -0.9 & 0.45 \end{bmatrix}.$$

Notice $\partial L / \partial W$ is a single column, shape $(3, 1)$ — matching W ’s own shape exactly, one gradient shared across the batch — while $\partial L / \partial X$ has one row per example, matching X ’s shape $(2, 3)$.

Also notice that every formula above needs X — the original input **forward** received, not anything it returned. **backward** receives `grad_output` as an argument, but nothing hands it X automatically: **forward** must save it, the same way you would save any other intermediate value a later step depends on. Store it as `self.x` during **forward**.

Why backward must write in place, not rebind

This is a subtlety that will not show up as a test failure until Chapter 10, several chapters from now — which makes it worth understanding clearly here, before it becomes a confusing bug much later with no obvious cause.

Chapter 9’s optimizer calls your `parameters()` method **once**, before training starts, to get the list of (`param`, `grad`) pairs it will update every step. Suppose **backward** computes the new gradient like this:

```
self.dW = ...some new array...
```

This line does not change the existing `self.dW` array — it creates a brand-new array and points the name `self.dW` at it instead. The optimizer, however, already captured a reference to the *old* array, back when `parameters()` was first called. Every subsequent **backward** call keeps creating new arrays and reassigning `self.dW` to them, while the optimizer keeps updating using the very first array it ever saw — which never changes again after that. Training runs, the loss barely moves, and nothing crashes to tell you why.

The fix: **pre-allocate `self.dW` and `self.db` once, in `__init__`**, using `np.zeros_like(self.W)` and `np.zeros_like(self.b)`. Then in **backward**, **write into that same array in place** rather than reassigning it — `self.dW[...] = ...` instead of `self.dW = ...`. The `[...]` on the left means “every position in the existing array,” so this mutates the array the optimizer already holds a reference to, instead of creating a new one it can no longer see. `self.db` needs the identical treatment.

This is genuinely easy to get wrong: both versions look almost identical in code, both pass Chapter 3’s own gradient-check test on their own (which only checks the *returned* array’s values, not whether the assignment inside was in-place), and the actual failure is only visible once Chapter 10 runs a real multi-step training loop. Get this right now, while it is one clear rule, rather than debug it later as a mysterious stall.

What to build

- Step 1.** Revisit `__init__` one more time: add `self.dW = np.zeros_like(self.W)` and `self.db = np.zeros_like(self.b)`, pre-allocated — not computed yet, just zeros the right shape — so **backward** always has an existing array to write into.
- Step 2.** Revisit **forward**: also store `self.x = x`, so **backward** can use it (see above).
- Step 3.** Implement **backward**: write `self.dW` and `self.db` **in place** (using the formulas above), and **return** $\partial L / \partial X$.
- Step 4.** Implement `parameters`: it must return a list of (`param`, `grad`) pairs so an optimizer can update them — `[(self.W, self.dW), (self.b, self.db)]`.

All four pieces already exist as stubs inside your `Linear` class from Chapters 1–2 — extend them, do not create a new class. Updated docstrings and TODOs for the whole class:

Python

```
def __init__(self, in_features: int, out_features: int) -> None:
    """Initialize the layer's weights, bias, and gradient buffers.

    Args:
        in_features (int): number of input features.
        out_features (int): number of output neurons.

    Sets:
        self.W (np.ndarray): weight matrix, shape
            (in_features, out_features). Xavier-initialized,
            not zeros (see "Weight initialization", Chapter 1).
        self.b (np.ndarray): bias vector, shape
            (out_features,). Initialized to zero.
        self.dW (np.ndarray): gradient buffer for self.W, same
            shape as self.W. Pre-allocated to zeros here so
            backward can write into it in place -- see "Why
            backward must write in place" above.
        self.db (np.ndarray): gradient buffer for self.b, same
            shape as self.b, pre-allocated the same way.
    """
    # TODO: create self.W, self.b (Chapter 1) and now also
    # self.dW, self.db, pre-allocated with np.zeros_like
    pass

def forward(self, x: np.ndarray) -> np.ndarray:
    """Compute this layer's output for a batch of inputs.

    Args:
        x (np.ndarray): input, shape (batch_size, in_features).

    Returns:
        np.ndarray: output, shape (batch_size, out_features).

    Sets:
        self.x (np.ndarray): the input, saved unchanged so
            backward can use it.
    """
    # TODO: implement the forward pass, using self.W and
    # self.b -- and now also store self.x = x
    pass

def backward(self, grad_output: np.ndarray) -> np.ndarray:
    """Compute gradients given the upstream gradient.

    Args:
        grad_output (np.ndarray): gradient of the loss with
```

```

        respect to this layer's output, shape (m, C).

Returns:
    np.ndarray: gradient of the loss with respect to
        this layer's input, shape (m, n).

Sets:
    self.dW (np.ndarray): overwritten in place (self.dW[...] = ...),
        not reassigned -- see "Why backward must write in
        place" above.
    self.db (np.ndarray): overwritten in place the same way.
"""
# TODO: write self.dW[...] and self.db[...] in place
# (sum over the batch axis for both, using self.x),
# and return dL/dX (NOT summed -- one row per example)
pass

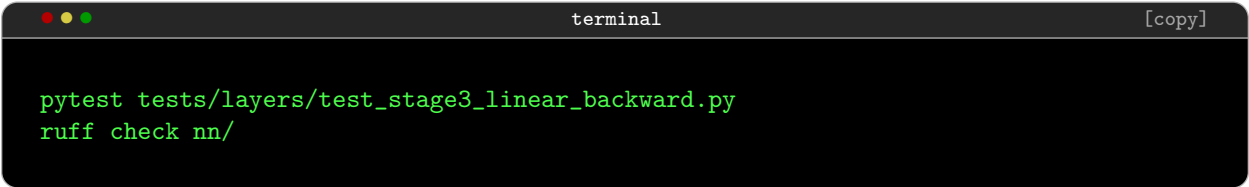
def parameters(self) -> list[tuple[np.ndarray, np.ndarray]]:
    """Return this layer's learnable parameters.

Returns:
    list[tuple[np.ndarray, np.ndarray]]: pairs of
        (parameter, gradient) --
        [(self.W, self.dW), (self.b, self.db)].
"""
# TODO: return [(self.W, self.dW), (self.b, self.db)]
pass

```

Validate before moving on

Run both — both must exit 0:



```

terminal [copy]

pytest tests/layers/test_stage3_linear_backward.py
ruff check nn/

```

`pytest` here uses **gradient checking**: it perturbs each input/parameter by a tiny amount, measures how much L changes, and compares that numerical estimate to your analytic **backward** output. A mismatch means a real bug — most often the batch-sum issue above. This is the strongest correctness check in the whole project; do not skip it even if your shapes look right, since a shape-correct but numerically wrong gradient will pass a shape check and still fail here.

Do not proceed to Chapter 4 until both pass.

Chapter 4 — ReLU: Forward and Backward

The idea

ReLU is the simplest activation, both directions:

$$\text{ReLU}(z) = \max(0, z), \quad \frac{d}{dz}\text{ReLU}(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}.$$

Forward zeroes out negative values and passes positive values through unchanged. Backward is a **mask**: wherever the forward input was positive, let the gradient through unchanged; wherever it was zero or negative, block it completely (multiply by 0).

Worked example

Let $\mathbf{z} = [-2, 0, 3]$, and suppose the upstream gradient (from whatever comes after ReLU) is $\partial L / \partial a = [0.5, -0.2, 0.1]$.

$\text{ReLU}(\mathbf{z}) = [0, 0, 3]$, $\text{mask} = [0, 0, 1]$ (using the convention that the boundary $z = 0$ counts as blocked).

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{a}} \odot \text{mask} = [0.5 \cdot 0, -0.2 \cdot 0, 0.1 \cdot 1] = [0, 0, 0.1].$$

Only the third entry survives — the first two had non-positive inputs during **forward**, so no gradient flows back through them, regardless of how large the upstream gradient was.

This means forward must remember which entries were positive. Store whatever you need (the input, or just the mask) as an instance attribute during **forward** so **backward** can use it later — this is true for every activation from here on, not just ReLU.

What to build

Step 1. Create `nn/activations/relu.py`:

Python

```
"""ReLU activation: zeroes out negative values."""

import numpy as np

from nn.module import Module

class ReLU(Module):
    """ReLU activation, applied elementwise: max(0, x)."""

    def forward(self, x: np.ndarray) -> np.ndarray:
        """Compute ReLU elementwise, and remember the mask.

        Args:
            x (np.ndarray): input, any shape.

        Returns:
```

```

        np.ndarray: max(0, x), elementwise, same shape as x.
    """
    # TODO: implement, and store whatever you need
    # from x for backward (e.g. a mask)
    pass

def backward(self, grad_output: np.ndarray) -> np.ndarray:
    """Compute gradients given the upstream gradient.

    Args:
        grad_output (np.ndarray): gradient of the loss with
            respect to this layer's output, same shape as
            the original input to forward.

    Returns:
        np.ndarray: gradient of the loss with respect to
            this layer's input, same shape as grad_output.
    """
    # TODO: implement, using what forward stored
    pass

```

Step 2. Add to `nn/activations/__init__.py`: `from .relu import ReLU`

Validate before moving on

Run both — both must exit 0:



```

pytest tests/activations/test_stage4_relu.py
ruff check nn/

```

`pytest` checks the worked example above, plus a gradient check, plus a batch of shape (m, C) to confirm the mask logic works elementwise across an entire matrix, not just a single row.

Chapter 5 — Sigmoid: Forward and Backward

The idea

Sigmoid has a famously clean derivative — one that reuses its own output:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)).$$

This means **backward** does not need to recompute anything from scratch — if **forward** already stored $a = \sigma(z)$, **backward** just computes $a(1 - a)$ and multiplies by the upstream gradient.

Worked example

Reuse $z = -1.3$ from Chapter 1. Suppose the upstream gradient is $\partial L / \partial a = 1$ (as if this were the very last step before the loss):

$$a = \sigma(-1.3) = 0.2142, \quad a(1 - a) = 0.2142 \times 0.7858 = 0.1683.$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot a(1 - a) = 1 \times 0.1683 = 0.1683.$$

Keep this number — Chapter 6 reuses it directly to demonstrate a much simpler shortcut.

What to build

Python

```
"""Sigmoid activation: squashes real values into (0, 1)."""

import numpy as np

from nn.module import Module

class Sigmoid(Module):
    """Sigmoid activation, applied elementwise: 1 / (1 + e^{-x})."""

    def forward(self, x: np.ndarray) -> np.ndarray:
        """Compute sigmoid elementwise, and remember the output.

        Args:
            x (np.ndarray): input, any shape.

        Returns:
            np.ndarray: sigmoid(x), elementwise, same shape as x.
        """
        # TODO: implement, and store the output --
        # backward reuses it directly
        pass

    def backward(self, grad_output: np.ndarray) -> np.ndarray:
        """Compute gradients given the upstream gradient.

        Args:
```

```

    grad_output (np.ndarray): gradient of the loss with
        respect to this layer's output, same shape as
        the original input to forward.

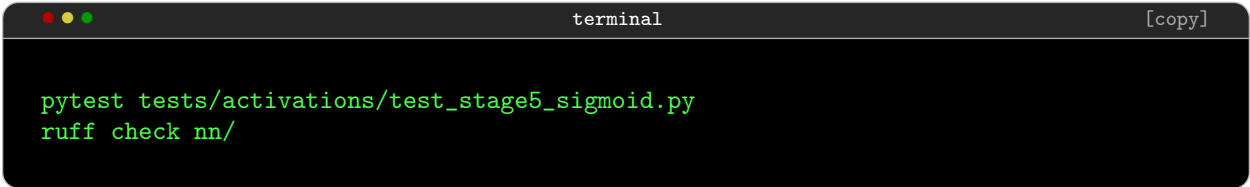
Returns:
    np.ndarray: gradient of the loss with respect to
        this layer's input, same shape as grad_output.
"""
# TODO: implement using a(1 - a)
pass

```

Save this as `nn/activations/sigmoid.py`, and add `from .sigmoid import Sigmoid` to `nn/activations/__init__.py`.

Validate before moving on

Run both — both must exit 0:



```

pytest tests/activations/test_stage5_sigmoid.py
ruff check nn/

```

`pytest` checks the worked value above, a gradient check, and — reusing the stability idea from Chapter 2 — that `forward` does not overflow on a large-magnitude input like $z = 1000$ or $z = -1000$.

Chapter 6 — Cross-Entropy Loss (Binary)

The idea

A loss takes predictions a (probabilities, from Sigmoid) and true labels y (0 or 1), and returns one scalar. Binary cross-entropy penalizes confident wrong answers heavily and confident right answers barely at all:

$$L = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(a_i) + (1 - y_i) \log(1 - a_i) \right].$$

Only one of the two terms is ever "active" per example, since y_i is 0 or 1 — when $y_i = 1$, the first term measures how far a_i is from 1; when $y_i = 0$, the second term measures how far a_i is from 0.

Its gradient with respect to the predictions:

$$\frac{\partial L}{\partial a_i} = -\frac{1}{m} \left(\frac{y_i}{a_i} - \frac{1 - y_i}{1 - a_i} \right).$$

A loss's backward looks slightly different

Every module so far takes `grad_output` as an argument to `backward` — the gradient flowing in from downstream. A loss is where the chain *starts*: there is nothing downstream of it, so its `backward` takes no argument and simply returns $\partial L / \partial a$ directly. Also note the signature difference in `forward`: a loss needs both the predictions *and* the true labels, not just one input.

Worked example, and a shortcut worth knowing

Reuse $a = 0.2142$ from Chapter 5, with true label $y = 1$, single example ($m = 1$):

$$L = -\log(0.2142) = 1.5410, \quad \frac{\partial L}{\partial a} = -\frac{1}{0.2142} = -4.6693.$$

Chain this through Sigmoid's backward from Chapter 5:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot a(1 - a) = -4.6693 \times 0.1683 = -0.7858.$$

Compare this to the much simpler expression $a - y = 0.2142 - 1 = -0.7858$ — identical. **When Sigmoid and binary cross-entropy are chained together, the combined gradient collapses to just $a - y$.** You are not implementing this shortcut directly (each module still computes its own honest `backward`) — but once you have both pieces built, chaining `CrossEntropyLoss.backward()` into `Sigmoid.backward(...)` should numerically equal $a - y$. Use this as a hand-checkable sanity test on top of the gradient checker.

Numerical stability: predictions of exactly 0 or 1

The formula above divides by a_i and by $1 - a_i$, and takes $\log(a_i)$ and $\log(1 - a_i)$. These are all well-defined for a_i strictly between 0 and 1 — but Sigmoid's output range is technically $(0, 1)$ *open*, and floating-point rounding can still land exactly on 0.0 or 1.0 for a sufficiently confident (or sufficiently wrong) prediction. If that happens, $\log(0)$ is $-\infty$ and division by 0 is undefined — your loss becomes `nan`, silently, often many training steps after the real cause.

Guard against this the same way you guarded softmax against overflow in Chapter 2: **clip predictions away from the exact boundary** before using them in either **forward** or **backward** — for example, `np.clip(predictions, 1e-12, 1 - 1e-12)`. This nudges an exact 0 or 1 a negligible distance inward, small enough not to affect any real computation, but large enough that log and division never see the actual boundary. Apply the same clipping in both methods, not just one — a value that is safe in **forward** but unclipped when reused in **backward** still breaks.

This matters more than it looks like it should. A `nan` loss appearing during Chapter 10's training loop, several chapters from now, is easy to misdiagnose as a learning-rate problem (Chapter 10's own troubleshooting section suggests exactly that first) when the real cause is an unclipped prediction here. Fixing it now, in the file where it actually originates, avoids chasing the wrong problem later.

What to build

Create `nn/losses/cross_entropy_loss.py`:

Python

```
"""Binary cross-entropy loss."""

import numpy as np

class CrossEntropyLoss:
    """Binary cross-entropy loss for a single output probability.

    Does not subclass Module -- see the note below.
    """

    def forward(self, predictions: np.ndarray, targets: np.ndarray) -> float:
        """Compute the average binary cross-entropy loss.

        Args:
            predictions (np.ndarray): predicted probabilities,
                shape (m,) or (m, 1). Clip away from exactly
                0 or 1 before use -- see "Numerical stability"
                above.
            targets (np.ndarray): true labels, same shape as
                predictions, values 0 or 1.

        Returns:
            float: the scalar loss, averaged over the batch.
        """
        # TODO: implement, and store whatever
        # backward needs. Clip predictions first,
        # e.g. np.clip(predictions, 1e-12, 1 - 1e-12)
        pass

    def backward(self) -> np.ndarray:
        """Compute the gradient of the loss w.r.t. predictions.

        Returns:
```

```

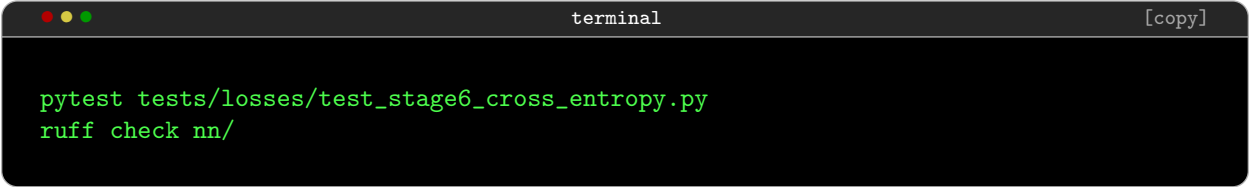
        np.ndarray: dL/da, same shape as the predictions
        passed to forward. Use the same clipped
        predictions here as in forward -- see
        "Numerical stability" above.
    """
    # TODO: implement -- no grad_output argument,
    # this is where the gradient chain starts.
    # Clip predictions the same way forward did.
    pass

```

Note this class does not subclass `Module` — its `forward` signature (predictions *and* targets) and its `backward` signature (no argument) do not match the `Module` contract from Chapter 0.5, and it has no `parameters()` to expose. Add `from .cross_entropy_loss import CrossEntropyLoss` to `nn/losses/__init__.py`.

Validate before moving on

Run both — both must exit 0:



```

pytest tests/losses/test_stage6_cross_entropy.py
ruff check nn/

```

`pytest` checks the worked value above, a gradient check on $\partial L / \partial a$, and the $a - y$ shortcut once chained with your Sigmoid from Chapter 5.

Chapter 7 — Softmax: Backward

Why this is harder than the other activations

Every activation so far is applied **independently** per element — ReLU and Sigmoid’s output i depends only on input i , so their backward is just an elementwise multiply. Softmax broke that independence in Chapter 2: every output in a row depends on *every* logit in that row through the shared denominator. That means softmax’s backward is not a simple elementwise multiply — each output’s gradient depends on *all* the inputs in its row, not just the matching one.

The Jacobian

For a single example, softmax maps a length- C vector to a length- C vector, so its derivative is a full $C \times C$ matrix (a Jacobian), not a single number per position:

$$\frac{\partial a_c}{\partial z_k} = a_c(\delta_{ck} - a_k), \quad \text{where } \delta_{ck} = \begin{cases} 1 & c = k \\ 0 & c \neq k \end{cases}.$$

Given the upstream gradient $\partial L / \partial \mathbf{a}$ (length C), the gradient with respect to the logits is this Jacobian applied to that vector, **per example**:

$$\frac{\partial L}{\partial z_k} = \sum_{c=1}^C \frac{\partial L}{\partial a_c} a_c(\delta_{ck} - a_k) = a_k \left(\frac{\partial L}{\partial a_k} - \sum_{c=1}^C \frac{\partial L}{\partial a_c} a_c \right).$$

For a batch, this must be computed **separately for each row** — there is no single matrix multiplication that reduces the whole batch to one Jacobian the way **Linear**’s backward could; conceptually, each example needs its own (C, C) Jacobian applied to its own upstream gradient. A loop over the batch dimension is the most direct way to implement this the first time, and is a completely reasonable place to start. A fully vectorized version (no explicit Python loop, using broadcasting the way Chapter 2’s **axis/keepdims** discussion did) is also possible and equally correct if you get there — neither approach is penalized over the other; only correctness (verified by the gradient check below) matters.

What to build

Python

```
def backward(self, grad_output: np.ndarray) -> np.ndarray:
    """Compute gradients given the upstream gradient.

    Args:
        grad_output (np.ndarray): gradient of the loss with
            respect to this layer's output, shape (m, C).

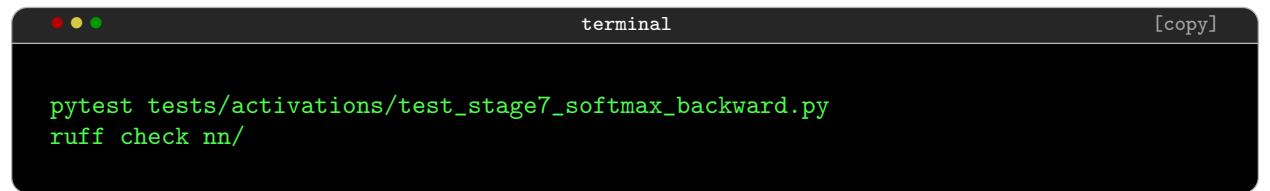
    Returns:
        np.ndarray: gradient of the loss with respect to
            this layer's input (the logits), shape (m, C).
    """
    # TODO: implement the per-example Jacobian formula
    # above. A loop over the batch dimension is a fine
    # starting point; a vectorized version is fine too
```

```
# if you find one -- either is correct as long as it  
# passes the gradient check below.  
pass
```

This goes in the `backward` method of the same `nn/activations/softmax.py` you created in Chapter 2.

Validate before moving on

Run both — both must exit 0:

A terminal window with a dark background. The title bar shows three colored dots (red, yellow, green) on the left, the word "terminal" in the center, and "[copy]" on the right. The terminal contains two lines of green text: "pytest tests/activations/test_stage7_softmax_backward.py" and "ruff check nn/".

```
pytest tests/activations/test_stage7_softmax_backward.py  
ruff check nn/
```

`pytest` is a pure gradient check against your Chapter 2 `forward` — there is no simple worked-by-hand example here the way there was for ReLU or Sigmoid, precisely because the Jacobian is a full matrix, not a single formula you can eyeball. Trust the gradient checker on this one.

Do not proceed until both pass. Chapter 8 gives you a second, independent way to sanity-check this same backward pass.

Chapter 8 — Categorical Cross-Entropy Loss

The idea

The multi-class counterpart to Chapter 6: given softmax probabilities $\mathbf{a}$ (one row per example, C columns) and true class labels as one-hot vectors $\mathbf{y}$, only the probability assigned to the *correct* class matters:

$$L = -\frac{1}{m} \sum_{i=1}^m \log(a_{i, \text{true class of } i}) = -\frac{1}{m} \sum_{i=1}^m \sum_{c=1}^C y_{i,c} \log(a_{i,c}).$$

Its gradient with respect to the probabilities is nonzero only at the true-class position:

$$\frac{\partial L}{\partial a_{i,c}} = -\frac{1}{m} \cdot \frac{y_{i,c}}{a_{i,c}}.$$

Worked example — and a way to check Chapter 7

Reuse the logits and softmax probabilities from Chapter 2: $\mathbf{z} = [-1.3, 0.3, 1.75]$, $\mathbf{a} = [0.0369, 0.1830, 0.7801]$. Suppose the true class is C (one-hot $\mathbf{y} = [0, 0, 1]$), single example:

$$L = -\log(0.7801) = 0.2484.$$

Just as Chapter 6 had a shortcut, so does this pairing — chaining `CategoricalCrossEntropyLoss.backward()` through your Chapter 7 `Softmax.backward()` should collapse to the strikingly simple:

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{a} - \mathbf{y} = [0.0369, 0.1830, 0.7801] - [0, 0, 1] = [0.0369, 0.1830, -0.2199].$$

This is the same shape of result as Chapter 6’s $a - y$, generalized from a single number to a vector. **Use this to check your Chapter 7 Jacobian implementation:** build a tiny `Softmax` → `CategoricalCrossEntropyLoss` pipeline with these exact numbers, chain the two `backward` calls, and confirm you get `[0.0369, 0.1830, -0.2199]`. If your Chapter 7 Jacobian has a bug, this is often where it becomes obvious — a wrong Jacobian rarely produces this exact clean form by accident.

The same clipping requirement as Chapter 6

$\log(a_{i,c})$ has the identical failure mode Chapter 6 covered: softmax’s output range is technically $(0, 1)$ open, but floating-point rounding can still produce an exact 0.0 for a very confident wrong prediction, and $\log(0)$ is $-\infty$. **Clip predictions the same way**, e.g. `np.clip(predictions, 1e-12, 1 - 1e-12)`, in both *forward* and *backward*, before taking any log or division. See Chapter 6’s “Numerical stability” section for the full reasoning — it applies here unchanged.

What to build

Create `nn/losses/categorical_cross_entropy_loss.py`, following the same non-Module pattern as Chapter 6:

Python

```
"""Categorical cross-entropy loss, for one-hot multi-class targets."""

import numpy as np
```

```

class CategoricalCrossEntropyLoss:
    """Categorical cross-entropy loss over C classes.

    Does not subclass Module -- see the note in Chapter 6.
    """

    def forward(self, predictions: np.ndarray, targets: np.ndarray) -> float:
        """Compute the average categorical cross-entropy loss.

        Args:
            predictions (np.ndarray): softmax probabilities,
                shape (m, C). Clip away from exactly 0 before
                use -- see "The same clipping requirement as
                Chapter 6" above.
            targets (np.ndarray): one-hot true labels, shape
                (m, C).

        Returns:
            float: the scalar loss, averaged over the batch.
        """
        # TODO: implement (targets as one-hot vectors).
        # Clip predictions first, e.g.
        # np.clip(predictions, 1e-12, 1 - 1e-12)
        pass

    def backward(self) -> np.ndarray:
        """Compute the gradient of the loss w.r.t. predictions.

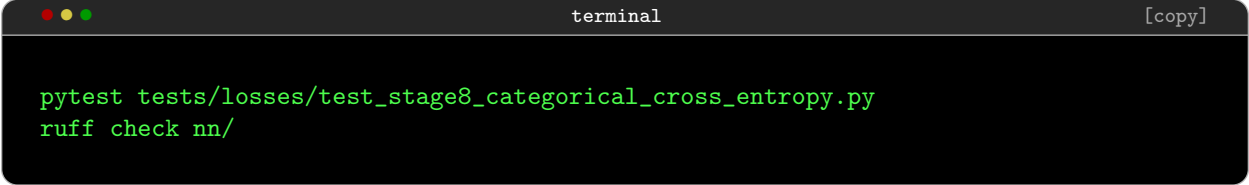
        Returns:
            np.ndarray: dL/da, shape (m, C), same shape as the
                predictions passed to forward. Use the same
                clipped predictions here as in forward.
        """
        # TODO: implement, using the same clipping as forward
        pass

```

Add from `.categorical_cross_entropy_loss` import `CategoricalCrossEntropyLoss` to `nn/losses/__init__.py`

Validate before moving on

Run both — both must exit 0:



```

terminal [copy]

pytest tests/losses/test_stage8_categorical_cross_entropy.py
ruff check nn/

```

`pytest` checks the worked value above, a gradient check, and the $\mathbf{a} - \mathbf{y}$ shortcut once chained with your Softmax backward from Chapter 7.

A note on binary vs. categorical, for this competition

The competition's target is binary (0 or 1). You can model this either way: **Sigmoid + CrossEntropyLoss** with a single output neuron ($C = 1$), or **Softmax + CategoricalCrossEntropyLoss** with two output neurons ($C = 2$, one-hot labels). These are mathematically equivalent — 2-class softmax reduces exactly to sigmoid — so either pipeline is valid for `main.py`. Pick one; do not feel obligated to wire up both into your final training script, though both must pass their own tests above.

Chapter 9 — SGD

The idea

Once every module exposes its parameters and gradients through `parameters()` (Chapter 3 did this for `Linear`), the optimizer’s job is almost mechanical: for every `(param, grad)` pair, nudge the parameter a small step in the direction that reduces the loss:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L,$$

where η is the learning rate. That is the entire update rule — no momentum, no adaptive per-parameter rates, just this, applied to every parameter the model exposes.

Why `zero_grad` matters

Gradients accumulate by design in this project (Chapter 3’s `backward` sets `self.dW`, but in general repeated `backward` calls without resetting first can add up). If you forget to reset gradients between training steps, step 2’s update includes leftover gradient from step 1, silently corrupting training without crashing anything — another bug category that will not announce itself loudly.

What to build

Create `nn/optim/sgd.py`:

```
Python

"""Stochastic gradient descent optimizer."""

import numpy as np

class SGD:
    """Vanilla SGD: param -= lr * grad, for every tracked parameter."""

    def __init__(
        self,
        parameters: list[tuple[np.ndarray, np.ndarray]],
        lr: float,
    ) -> None:
        """Store the parameters to update and the learning rate.

        Args:
            parameters (list[tuple[np.ndarray, np.ndarray]]):
                (param, grad) pairs, as returned by a module's
                parameters() method.
            lr (float): learning rate, the step size eta.
        """
        # TODO: store what you need
        pass

    def step(self) -> None:
        """Apply one update step to every tracked parameter.
```

```

    Updates every param in place using its current grad,
    following param -= lr * grad. Does not return anything.
    """
    # TODO: for every (param, grad) pair,
    # apply the update rule above, in place
    pass

def zero_grad(self) -> None:
    """Reset every tracked parameter's gradient to zero.

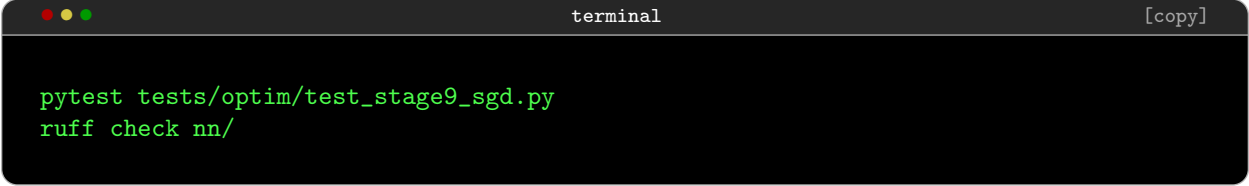
    Does not return anything.
    """
    # TODO: reset every parameter's stored
    # gradient back to zero
    pass

```

SGD is not a Module either — it does not compute a forward or backward pass, it only reads gradients that other modules already computed and updates parameters in place. Add `from .sgd import SGD` to `nn/optim/__init__.py`.

Validate before moving on

Run both — both must exit 0:



```

pytest tests/optim/test_stage9_sgd.py
ruff check nn/

```

`pytest` checks that a single `step()` moves a parameter in the correct direction (opposite the gradient) by the correct amount on a small toy example, and that `zero_grad()` actually zeroes every stored gradient, not just some of them.

Chapter 10 — `main.py`: Putting It All Together

The idea

Every piece now independently passes its own tests. This chapter is where you assemble them into an actual trained model, on a small toy dataset, to confirm the whole chain — forward, backward, and the optimizer step — genuinely works together before you touch the real competition data.

Build, in `main.py`:

1. A `Linear` layer, a `Sigmoid` (or `Softmax`), and either `CrossEntropyLoss` or `CategoricalCrossEntropyLoss` — matching pair, per the note at the end of Chapter 8.
2. An SGD optimizer, given the parameters your `Linear` layer exposes.
3. A training loop: for some number of epochs, run **forward** through every module in order, compute the loss, run **backward** through every module in *reverse* order (loss first, then the activation, then `Linear` last), call `optimizer.step()`, then `zero_grad()` before the next iteration.

Required interface

The provided `tests/test_stage10_training_converges.py` imports these exact symbols from `main.py` — expose them with exactly these names and signatures, or the provided test will fail to import your code regardless of whether your network itself is correct:

- `toy_data()` -> `tuple[np.ndarray, np.ndarray]` — returns `(X, y)`, the toy dataset described below.
- `train(epochs: int = 4000, lr: float = 1.0, seed: int = 0)` -> `list[float]` — runs the training loop and returns the loss recorded at every epoch, in order.
- `accuracy(loss_history: list[float] = None)` -> `float` — a helper that reports final classification accuracy. (The exact relationship between this and `train`'s return value — whether `accuracy` re-runs training itself or expects to be called after — is yours to design; the test only checks the final number this returns.)

Also include a `if __name__ == "__main__":` block that calls `train()` and prints progress — this is what runs when a human executes `python main.py` directly, separate from what `pytest` imports and calls programmatically.

If your script also loads real data later for your actual competition submission, that path is relative to the submission root — **[TBD: exact dataset path, e.g. `data/train.csv`, to be confirmed in a follow-up]**. The dataset itself is provided in the repository; you do not fetch, generate, or place it yourself.

What a healthy run looks like

Do not use XOR. It is the standard toy example in most neural network tutorials, but those tutorials assume a network with at least one *hidden* layer. This project's network is explicitly a single layer throughout (Chapter 0.5) — one `Linear`, one activation, nothing stacked in between — which makes it mathematically a linear classifier (logistic or softmax regression), no matter how it is initialized or trained. XOR is the textbook example of a function that is **not linearly separable**:

no single straight-line decision boundary can separate its classes, by design. A linear classifier cannot learn XOR, full stop — not with more epochs, not with a different learning rate, not with a bug-free implementation. If you use it, training loss will plateau around $-\ln(0.5) \approx 0.693$ and accuracy will sit near 50%, and everything in the troubleshooting list below will send you looking for a bug that does not exist.

Use a small **linearly separable** toy set instead — the AND gate is the standard choice for exactly this kind of single-layer sanity check:

$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

This is still a genuine test of your whole pipeline — four points, not a single-class trivial case — and it is actually solvable by the network you built, which is the entire point of this check: confirming your *implementation* works, not testing the limits of what a single linear layer can represent.

Print the loss every so often. You should see it **decrease steadily** toward roughly 0, with accuracy climbing to 100%, not oscillate wildly or plateau far above 0.

If it does not:

- **Loss is nan almost immediately** — almost always a numerical stability issue (Chapters 2, 5, 6, or 8 — softmax overflow, sigmoid overflow, or an unclipped prediction hitting $\log(0)$ in either loss), or a learning rate so large the parameters are overshooting catastrophically. Try a much smaller learning rate first to isolate which; if the **nan** persists even at a tiny learning rate, it is a stability bug in one of those four chapters, not this chapter.
- **Loss barely moves at all** — suspect the training loop itself before suspecting your modules: are you calling `zero_grad()` at the right point (before or after `step()`, but consistently every iteration)? Is `optimizer.step()` actually being called? Every module already passed its own gradient check in earlier chapters, so a stuck loss more often means a wiring bug here, not a math bug in a module you already validated. If you implemented **backward** by *rebinding* `self.dW/self.db` instead of writing in place (Chapter 3’s warning), this is exactly where that bug surfaces — go back and check.
- **Loss decreases, then increases, then decreases erratically** — usually the learning rate is too high. Lower it before suspecting anything else.
- **Loss decreases smoothly but plateaus well above 0, accuracy stuck near 50–75%** — if you are using AND as recommended above, this should not happen; if it does, double check you are using AND and not XOR (see above) before suspecting anything else.

Validate

Run all three — all must exit 0:

A terminal window with a dark background and light green text. The title bar shows three colored dots (red, yellow, green) on the left and the word 'terminal' in the center. On the right side of the title bar is a '[copy]' button. The terminal content consists of three lines of code:

```
pytest tests/test_stage10_training_converges.py
ruff check nn/
ruff check main.py
```

The `pytest` run asserts your `main.py` pipeline’s loss decreases over a fixed number of epochs and crosses a minimum accuracy threshold — on the **provided toy dataset**, not your real competition data. This test exists to check your training loop and module wiring in isolation from any question about how well you have tuned things on the actual competition data; it is the first test in the whole project that does not test one file in isolation, and is the payoff for every earlier chapter passing. The two `ruff` runs check documentation on everything you have written this whole guide — `main.py` included, not just `nn/`, since Chapter 0’s documentation requirements cover every file you write.

Once this passes, the same structure — swap the toy AND-gate data for the real dataset (path above, once confirmed), keep everything else identical — is what you adapt for your actual competition submission.

Shapes at a glance

Every bug in this project shows up as a shape mismatch before it shows up as a wrong number — so it is worth having the full pipeline’s shapes in one place, for a batch of m examples, n input features, and C classes:

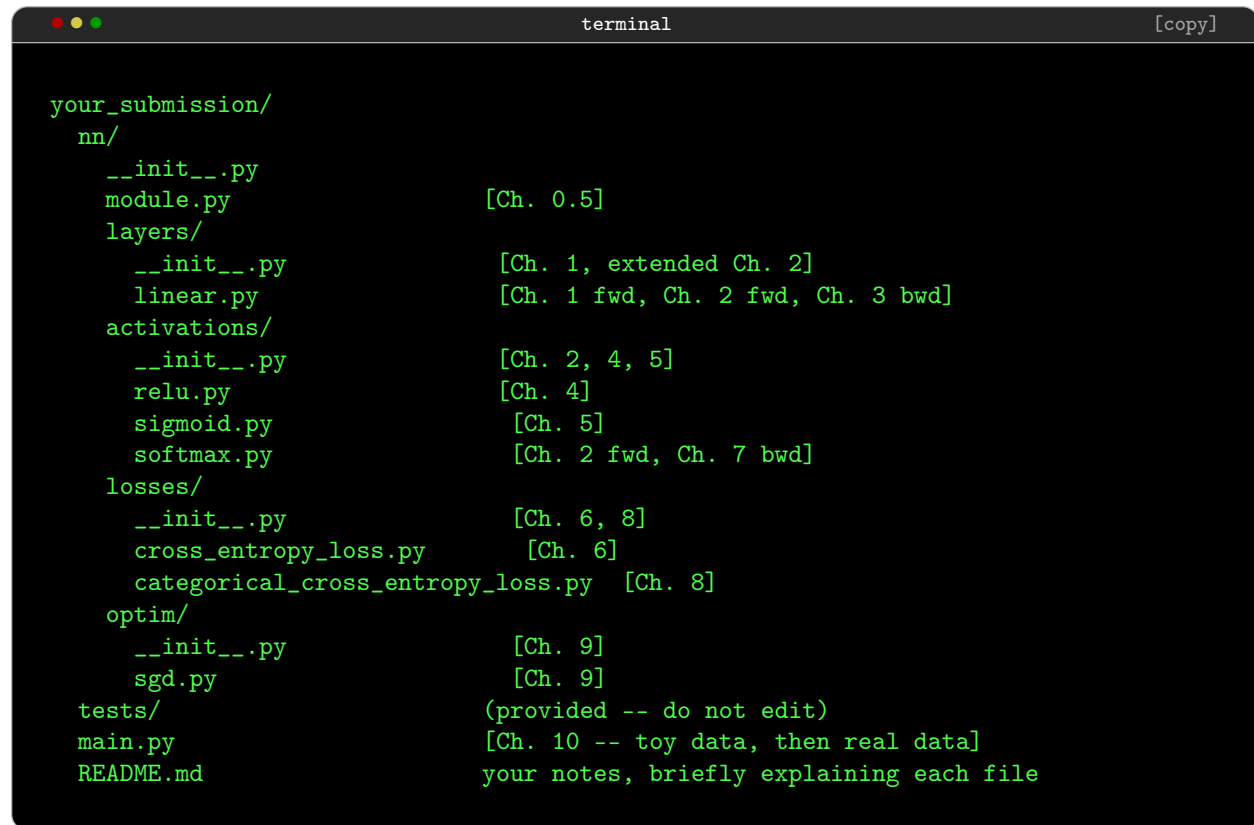
Stage	What it computes	Output shape
Input	the raw batch X	(m, n)
Linear layer	$Z = XW^T + \mathbf{b}$	(m, C) — C logits per example
Softmax activation	$A = \text{softmax}(Z)$, row-wise	(m, C) — shape unchanged
Cross-entropy loss	one scalar summarizing all m predictions	$()$ — a single number

Two things worth internalizing from this table. First, **the activation never changes shape** — whatever shape goes in comes out identical; only *values* change, not *dimensions*. This holds for softmax too, but with a caveat from Chapter 2: unlike a fully elementwise activation, each softmax output within a row depends on every logit in that same row, not just the matching input — the shape is preserved, but the independence is not. Second, **the loss is the only stage that collapses a dimension** — everything before it tracks C values per example, and the loss reduces all of that down to the single number you actually optimize. When you implement `backward()` later, this table runs in reverse: the loss’s backward is the first place a gradient’s shape *expands* again, from one scalar back out to (m, C) , matching the shape it collapsed at.

The parameters themselves have their own fixed shapes, independent of batch size: W is (C, n) and $\mathbf{b}$ is $(C,)$, no matter whether you are processing 1 example or 10,000 — only X ’s first dimension changes with batch size. If a shape assertion fails while you are debugging, check parameter shapes first (they should never change) before suspecting the data shapes (which do, batch to batch).

What your finished submission looks like

This is the completed structure, every file now covered by a chapter above:



```
terminal [copy]

your_submission/
  nn/
    __init__.py           [Ch. 0.5]
    module.py
    layers/
      __init__.py         [Ch. 1, extended Ch. 2]
      linear.py           [Ch. 1 fwd, Ch. 2 fwd, Ch. 3 bwd]
    activations/
      __init__.py         [Ch. 2, 4, 5]
      relu.py             [Ch. 4]
      sigmoid.py          [Ch. 5]
      softmax.py          [Ch. 2 fwd, Ch. 7 bwd]
    losses/
      __init__.py         [Ch. 6, 8]
      cross_entropy_loss.py [Ch. 6]
      categorical_cross_entropy_loss.py [Ch. 8]
    optim/
      __init__.py         [Ch. 9]
      sgd.py              [Ch. 9]
  tests/                  (provided -- do not edit)
  main.py                 [Ch. 10 -- toy data, then real data]
  README.md              your notes, briefly explaining each file
```

Every file above is now covered by some chapter in this guide — `linear.py` across three chapters (built incrementally: single neuron, many neurons, then its backward pass), `softmax.py` across two (forward in Chapter 2, its harder backward in Chapter 7), everything else in one chapter each. If a file above still has a bare `pass` in it and you cannot find the chapter that covers it, check the table of contents — nothing was left unaddressed.

Gradient shapes at a glance

The forward shapes table earlier in this guide tracked $X \rightarrow Z \rightarrow A \rightarrow L$. Backpropagation runs the same pipeline in reverse, and — reassuringly — every gradient has *exactly the same shape* as the value it is a gradient of:

Gradient	Shape
$\partial L / \partial L$ (the starting point, implicitly 1)	()
$\partial L / \partial A$	(m, C) — same as A
$\partial L / \partial Z$	(m, C) — same as Z
$\partial L / \partial W$	(C, n) — same as W
$\partial L / \partial \mathbf{b}$	$(C,)$ — same as $\mathbf{b}$
$\partial L / \partial X$	(m, n) — same as X

This is not a coincidence — it is a general fact of calculus that the gradient of a scalar with respect to any tensor has the same shape as that tensor. Use it as a cheap, constant sanity check while implementing every **backward** method in this guide: if the shape you are about to return does not match the shape of what you took the gradient of, you already have a bug, before you even check the values.